

DATA 260 Homework 1 Report

GITHUB REPOSITORY: <https://github.com/rutujaste06/data260-5825>

Student ID- **020065825**

SID4- **5825**

PORT_BASE - **8425**

PREFIX - **s5825**

SEED - **5825**

VERIFY_SEED – **265825**

DOMAIN_ID - **1**

DOMAIN - Clinical trial listings

HARDWARE - 13th Gen Intel Core i7-13700HX (2.10 GHz), 16GB RAM, NVIDIA GeForce RTX 4070 Laptop GPU (8GB)

LOCAL MODEL USED: **qwen3:8b (Ollama)**

TAGGED COMMIT HASH: **714d9fb7436bf5b5e06349ad43ab117b65cc6355**

OUTPUTS:

PART I:

created DOMAIN_SCHEMA.md defining the clinical trial entity's fields (Trial Title, NCT Number, Submitter Email, Trial Description, Trial Phase, Terms Agreement) and the four Trial Phase category values (Phase I, Phase II, Phase III, Phase IV).

```
DOMAIN_SCHEMA.md > # Domain Schema: Clinical Trial Listings > ## Category Values — Trial Phase
1 # Domain Schema- Clinical Trial Listings
2
3 ## Entity Fields
4
5 | Field | Type | Description |
6 |-----|-----|-----|
7 | Trial Title | Text (required) | Primary field – full name of the clinical trial |
8 | NCT Number | Text (required) | Secondary field – unique trial registry ID (format: NCT + 8 digits) |
9 | Submitter Email | Email (required) | Email of the person submitting the entry |
10 | Trial Description | Textarea (required) | Summary of the trial's purpose and methodology (must exceed 25 characters) |
11 | Trial Phase | Dropdown (required) | Category field, one of four fixed phase values |
12 | Terms Agreement | Checkbox (required) | Confirmation that the submitter agrees to terms and conditions |
13
14 ## Category Values – Trial Phase
15
16 - Phase I
17 - Phase II
18 - Phase III
19 - Phase IV
```

Built an HTML form for clinical trial listings with fields for Trial Title, NCT Number, Submitter Email, Trial Description, Trial Phase, and terms agreement checkbox.

```
EXPLORER
  DATA260-5825
    DOMAIN_SCHEMA.html
    index.html
    README.md
    script.js

index.html
  1 <!DOCTYPE html>
  2 <html lang="en">
  3 <head>
  4   <meta charset="UTF-8">
  5   <title>H4-Rutuja</title>
  6 </head>
  7 <body>
  8   <h1>Clinical Trial Listings</h1>
  9
 10   <form id="trialForm">
 11     <label for="trialTitle">Trial Title</label>
 12     <input type="text" id="trialTitle" name="trialTitle" placeholder="Enter the full title of the clinical trial" required autofocus>
 13   </form>
 14
 15   <label for="nctNumber">NCT Number</label>
 16   <input type="text" id="nctNumber" name="nctNumber" placeholder="e.g., NCT02345678" required>
 17 </form>
 18
 19   <label for="submitterEmail">Submitter Email</label>
 20   <input type="email" id="submitterEmail" name="submitterEmail" placeholder="Enter your email address" required>
 21 </form>
 22
 23   <label for="trialDescription">Trial Description</label>
 24   <textarea id="trialDescription" name="trialDescription" placeholder="Describe the trial's purpose, methodology, and goals (minimum 25 characters)" required></textarea>
 25 </form>
 26
 27   <label for="trialPhase">Trial Phase</label>
 28   <select id="trialPhase" name="trialPhase" required>
 29     <option value="">Select Phase</option>
 30     <option value="Phase I">Phase I</option>
 31     <option value="Phase II">Phase II</option>
 32     <option value="Phase III">Phase III</option>
 33     <option value="Phase IV">Phase IV</option>
 34   </select>
 35 </form>
 36
 37   <input type="checkbox" id="agreeForm" name="agreeForm" required>
 38   <label for="agreeForm">I agree to the terms and conditions</label>
 39 </form>
 40
 41   <button type="submit">Submit Trial Listing</button>
 42 </form>
 43 <script src="script.js"></script>
 44 </body>
 45 </html>
```

File C:/Users/vivek/Desktop/Rutuja%20docs/DATA%20260/data260-5825/index.html

Clinical Trial Listings

Trial Title:

NCT Number:

Submitter Email:

Trial Description:

Trial Phase:

☒ I agree to the terms and conditions.

PART II: Javascript

Verified the alert pop ups if validation fails

```
index.html
  1 <script>
  2   const createSubmissionCounter = () => {
  3     let count = 0;
  4     return () => {
  5       count++;
  6       return count;
  7     };
  8   };
  9
 10   const trackSubmission = createSubmissionCounter();
 11
 12   const validateForm = (event) => {
 13     event.preventDefault();
 14
 15     const description = document.getElementById("trialDescription").value;
 16     const agreeChecked = document.getElementById("agreeForm").checked;
 17
 18     if (description.length <= 25) {
 19       alert("Trial Description must be more than 25 characters.");
 20       return;
 21     }
 22
 23     if (!agreeChecked) {
 24       alert("You must agree to the terms and conditions.");
 25       return;
 26     }
 27
 28     const formData = {
 29       trialTitle: document.getElementById("trialTitle").value,
 30       nctNumber: document.getElementById("nctNumber").value,
 31       submitterEmail: document.getElementById("submitterEmail").value,
 32       trialDescription: description,
 33       trialPhase: document.getElementById("trialPhase").value,
 34       agreeForm: agreeChecked
 35     };
 36
 37     const jsonString = JSON.stringify(formData);
 38     console.log("Form data as JSON string", jsonString);
 39
 40     const parsedData = JSON.parse(jsonString);
 41     console.log("trialTitle, submitterEmail", parsedData);
 42     console.log("trialTitle", trialTitle);
 43     console.log("Submitter Email", submitterEmail);
 44
 45     const updatedData = { ...parsedData, submissionDate: new Date().toDateString() };
 46     console.log("Updated data with submission date", updatedData);
 47
 48     const submissionCount = trackSubmission();
 49     console.log("Submission count", submissionCount);
 50   };
 51
 52   document.getElementById("trialForm").addEventListener("submit", validateForm);
```

Clinical Trial Listings

Trial Title:

NCT Number:

Submitter Email:

Trial Description:

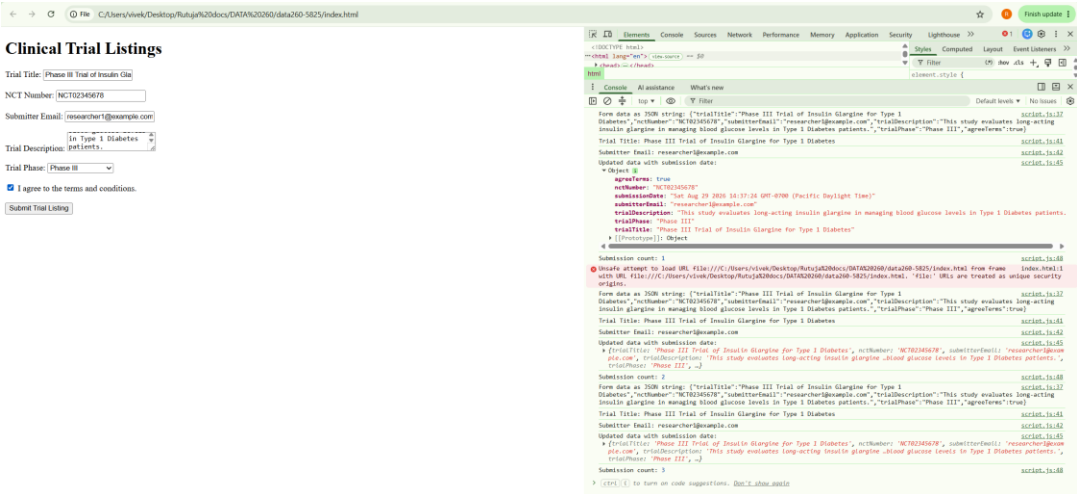
Trial Phase:

☒ I agree to the terms and conditions.

☐ Please check this box if you want to proceed.



Validated the console outputs: Console output showing JSON string, destructured values, updated object with submissionDate, and incrementing submission counts



Docker file creation

Docker file content:

DEPLOYMENT:

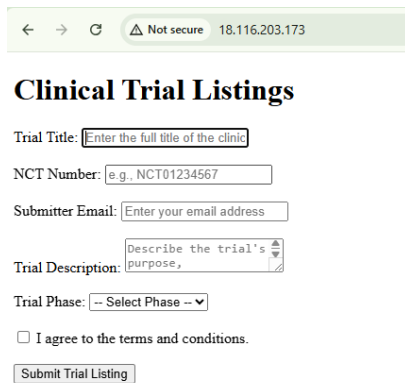
Docker Image Tagged and Pushed to Amazon ECR:

docker tag command successfully tagging the local image and

docker push command pushing all layers to the clinical-trial-app repository in Amazon ECR

```
C:\Users\vivek>docker tag clinical-trial-app:latest 093468662825.dkr.ecr.us-east-2.amazonaws.com/clinical-trial-app:latest
C:\Users\vivek>docker push 093468662825.dkr.ecr.us-east-2.amazonaws.com/clinical-trial-app:latest
The push refers to repository [093468662825.dkr.ecr.us-east-2.amazonaws.com/clinical-trial-app]
c072f83ab0c3: Pushed
44136fa355b3: Pushed
d94291c26261: Pushed
55afa1ecc21d: Pushed
0ea935727878: Pushed
1ed8e39a7434: Pushed
0d62d88506ba: Pushed
8cdf4f23778: Pushed
da4dea1b00af: Pushed
fb597529c916: Pushed
5acef8123c6a: Pushed
0938e7230cea: Pushed
latest: digest: sha256:00bd2738528866ac42d7749aa8e2e987795f7af464a151d893ba553cff42d21e size: 856
```

Application running on the public IP address from ECS



The screenshot shows a web browser window with the address bar displaying "18.116.203.173". The page title is "Clinical Trial Listings". The form contains the following fields and controls:

- Trial Title:
- NCT Number:
- Submitter Email:
- Trial Description: (with a small icon to the right)
- Trial Phase:
- ☐ I agree to the terms and conditions.
-

Pipeline: Planner, Reviewer, and Finalizer Output: Console transcript from running python agents_demo.py, showing the Planner agent's initial proposed tags and summary, the Reviewer agent's evaluation (which confirmed the Planner's output was already accurate), and the Finalizer's validated "Publish output" JSON containing exactly 3 tags and a summary within the 25-word limit.

1. Command used: python agents_demo.py
2. Console showing: Planner output, Reviewer output, Finalized output

```

@ README.md X  reproducible_run_instructions.md  Dockerfile  DOMAIN_SCHEMA.md  agents_demo.py code X  hw1_demo.py code 1  verify.py code  test_agent_reviewer.py code 1  run_experiment.py
code > agents_demo.py > ...
1 import json
2 import argparse
3 import time
4 from langchain_ollama import ChatOllama
5
6 # Sample input - clinical trial domain (used only if run without --input)
7 sample_title = "Phase II Study of Metformin for Type 2 Diabetes"
8 sample_content = {
9     "This trial evaluates the efficacy and safety of Metformin in managing "
10     "blood glucose levels among adult patients diagnosed with Type 2 Diabetes. "
11     "Participants will be monitored over a 12-week period, with primary "
12     "endpoints focused on HbA1c reduction and secondary endpoints assessing "
13     "adverse events and patient-reported outcomes."
14 }
15
16 def planner_agent(title, content):
17     prompt = """You are a Planner agent. Given a title and content, propose:
18     1. Exactly 3 topical tags (short phrases) that best describe the content.
19     2. A summary of AT MOST 25 words. Count your words carefully - do not exceed 25 words under any circumstances. Prioritize a complete, grammatically correct sentence within that limit..
20
21     Respond ONLY in valid JSON format like this:
22     [{"tags": ["tag1", "tag2", "tag3"], "summary": "your summary here"}]
23
24     Title: {title}
25     Content: {content}
26     """
27     response = llm.invoke(prompt)
28     return response.content
29
30 def reviewer_agent(title, content, planner_output):
31     prompt = """You are a Reviewer agent. You will review the Planner's proposed
32     tags and summary for accuracy and quality, given the original title and content.
33
34     If the tags or summary need improvement, provide corrected versions.
35     If they are already good, you may keep them the same.
36     The summary must be AT MOST 25 words - count carefully and do not exceed this limit.
37
38     Respond ONLY in valid JSON format like this:
39     [{"tags": ["tag1", "tag2", "tag3"], "summary": "your summary here"}]
40
41     Title: {title}
42     Content: {content}
43
44     Planner's proposal: {planner_output}
45     """
46     response = llm.invoke(prompt)
47     return response.content
48
49 def finalizer_agent(reviewer_output):
50     try:
51         parsed = json.loads(reviewer_output)
52     except json.JSONDecodeError:
53         parsed = {"tags": [], "summary": ""}
54
55     summary = parsed.get("summary", "")
56     words = summary.split()
57     if len(words) > 25:
58         summary = " ".join(words[:25])
59
60

```

```

@ README.md  reproducible_run_instructions.md  Dockerfile  DOMAIN_SCHEMA.md  agents_demo.py code X
code > agents_demo.py > ...
52 def finalizer_agent(reviewer_output):
53
54     final_output = {
55         "tags": parsed.get("tags", [])[:3],
56         "summary": summary
57     }
58     return final_output
59
60 if __name__ == "__main__":
61     parser = argparse.ArgumentParser()
62     parser.add_argument("--input", type=str, default="./reports/hw01/cases/nondeterminism_input.json")
63     parser.add_argument("--temp", type=float, default=0.7)
64     args = parser.parse_args()
65
66     with open(args.input, "r") as f:
67         data = json.load(f)
68         title = data["title"]
69         content = data["content"]
70
71     llm = ChatOllama(model="qwen3:8b", temperature=args.temp)
72
73     start_time = time.time()
74
75     print("=== Planner Agent ===")
76     planner_output = planner_agent(title, content)
77     print(planner_output)
78     print()
79
80     print("=== Reviewer Agent ===")
81     reviewer_output = reviewer_agent(title, content, planner_output)
82     print(reviewer_output)
83     print()
84
85     print("=== Finalized Publish Output ===")
86     final_output = finalizer_agent(reviewer_output)
87     print(json.dumps(final_output, indent=2))
88
89     end_time = time.time()
90     latency_ms = (end_time - start_time) * 1000
91
92     print(f"\nLatency: {latency_ms:.2f} ms")

```

```

(venv) C:\Users\vivek\Desktop\Rutuja docs\DATA 260\data260-5825>python agents_demo.py
=== Planner Agent ===
{"tags": ["Phase II Trial", "Metformin", "Type 2 Diabetes"], "summary": "Phase II trial evaluates Metformin's efficacy and safety in Type 2 Diabetes over 12 weeks, focusing on HbA1c reduction and adverse events."}

=== Reviewer Agent ===
{"tags": ["Phase II Trial", "Metformin", "Type 2 Diabetes"], "summary": "Phase II trial evaluates Metformin's efficacy and safety in Type 2 Diabetes over 12 weeks, focusing on HbA1c reduction and adverse events."}

=== Finalized Publish Output ===
{
  "tags": [
    "Phase II Trial",
    "Metformin",
    "Type 2 Diabetes"
  ],
  "summary": "Phase II trial evaluates Metformin's efficacy and safety in Type 2 Diabetes over 12 weeks, focusing on HbA1c reduction and adverse events."
}

```

Q1. The 3 final tags you obtained. –

"Phase II Trial", "Metformin", "Type 2 Diabetes"

Q2. The final summary (≤25 words).

Phase II trial evaluates Metformin's efficacy and safety in Type 2 Diabetes over 12 weeks, focusing on HbA1c reduction and adverse events

Q3. Did the Reviewer agent change anything? (yes/no +explanation)

No, As in my prompt design, I told the Reviewer it could either correct the Planner's output or keep it the same if it was already good and hence the Reviewer evaluated the Planner's tags ("Phase II Trial", "Metformin", "Type 2 Diabetes") and summary, and determined they were already accurate and well-formed, so it returned them unchanged. Given the prompt below:

"def reviewer_agent(title, content, planner_output): prompt = f""You are a Reviewer agent. You will review the Planner's proposed tags and summary for accuracy and quality, given the original title and content."

Explanation of each step: Planner, Reviewer , Finalizer:

Planner: Takes the input title as well as content and proposes an initial set of exactly 3 tags along with a draft summary which is less than 25 words. It returns the output in JSON format.

Reviewer: Takes the planner's proposed tags and summary, along with the original title/content, and checks them for accuracy and quality. It corrects them if required or keeps them unchanged if they're already good.

Finalizer: Takes the Reviewer's output, parses it, and makes sure there are exactly 3 tags and the summary is not too long that is not more than 25 words. It then produces the final validated "Publish output" in json format which is the final clean result.

Measuring Non-Determinism (Part 3):

Running the Agent 40 Times (20 runs at Temperature 0.7, 20 runs at Temperature 0.0):

```

1 client.py code 1  verify.py code  test_agent_review.py code 1  run_experiments.py code X  analyze_results.py code  index
code > run_experiments.py > run_single
1 import subprocess
2 import json
3 import time
4 import statistics
5 import csv
6 import os
7 import sys
8
9 RESULTS_DIR = "../reports/hw01/raw"
10 os.makedirs(RESULTS_DIR, exist_ok=True)
11
12 def run_single(temp, run_number):
13     start = time.time()
14     result = subprocess.run(
15         [sys.executable, "agents_demo.py", "--temp", str(temp)],
16         capture_output=True, text=True
17     )
18     end = time.time()
19     latency_ms = (end - start) * 1000
20
21     output = result.stdout
22     try:
23         marker = "Finalized Publish Output "
24         json_start = output.index(marker) + len(marker)
25         json_text = output[json_start:].split("Latency:")[0].strip()
26         parsed = json.loads(json_text)
27         tags = parsed.get("tags", [])
28         summary = parsed.get("summary", "")
29     except Exception as e:
30         tags = []
31         summary = ""
32     print(f" [Warning] failed to parse run {run_number} at temp {temp}: {e}")
33     print(f" [DEBUG] STDOUT: {output}")
34     print(f" [DEBUG] STDERR: {result.stderr}")
35
36     return {
37         "run_number": run_number,
38         "temperature": temp,
39         "tags": tags,
40         "summary": summary,
41         "latency_ms": round(latency_ms, 2)
42     }
43
44 def run_batch(temp, count):
45     results = []
46     for i in range(1, count + 1):
47         print(f"Running temp={temp}, run {i}/{count}...")
48         r = run_single(temp, i)
49         results.append(r)
50     print(f" Tags: {r['tags']}, Latency: {r['latency_ms']}ms")
51     return results
52
53 if __name__ == "__main__":
54     all_results = []
55
56     print("=== Starting 20 runs at temperature 0.7 ===")
57     results_07 = run_batch(0.7, 20)
58     all_results.extend(results_07)
59
60     print("\n=== Starting 20 runs at temperature 0.0 ===")
61     results_00 = run_batch(0.0, 20)
62     all_results.extend(results_00)
63

```

```

if __name__ == "__main__":
    all_results = []

    print("=== Starting 20 runs at temperature 0.7 ===")
    results_07 = run_batch(0.7, 20)
    all_results.extend(results_07)

    print("\n=== Starting 20 runs at temperature 0.0 ===")
    results_00 = run_batch(0.0, 20)
    all_results.extend(results_00)

    with open(f"{RESULTS_DIR}/all_runs.json", "w") as f:
        json.dump(all_results, f, indent=2)

    with open(f"{RESULTS_DIR}/all_runs.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["run_number", "temperature", "tags", "summary", "latency_ms"])
        writer.writeheader()
        for r in all_results:
            writer.writerow({"run_number": r["run_number"], "temperature": r["temperature"], "tags": "; ".join(r["tags"])})

    print("\nAll 40 runs complete. Results saved to reports/hw01/raw/")

```



```

11ent.py code 1  verify code  test_agent_review.py code 1  run_experiments.py code  analyze_results.py code X
code > analyze_results.py > ...
4
5 with open("../reports/hw01/raw/all_runs.json", "r") as f:
6     all_results = json.load(f)
7
8 def analyze_temp(results, temp_label):
9     tag_sets = [frozenset(r["tags"]) for r in results]
10    distinct_sets = set(tag_sets)
11
12    tag_counts = Counter()
13    for r in results:
14        for tag in set(r["tags"]):
15            tag_counts[tag] += 1
16
17    total_runs = len(results)
18    tags_in_all = [tag for tag, count in tag_counts.items() if count == total_runs]
19    tags_in_exactly_one = [tag for tag, count in tag_counts.items() if count == 1]
20
21    latencies = sorted(r["latency_ms"] for r in results)
22    p50 = statistics.median(latencies)
23    p95 = latencies[int(len(latencies) * 0.95) - 1]
24    p99 = latencies[int(len(latencies) * 0.99) - 1]
25
26    print(f"==== Temperature {temp_label} ===")
27    print(f"Distinct tag sets: {len(distinct_sets)}")
28    print(f"Tags in all {total_runs} runs: {tags_in_all}")
29    print(f"Tags in exactly 1 run: {tags_in_exactly_one}")
30    print(f"Latency p50: {p50:.2f} ms")
31    print(f"Latency p95: {p95:.2f} ms")
32    print(f"Latency p99: {p99:.2f} ms")
33
34    return {
35        "distinct_tag_sets": len(distinct_sets),
36        "tags_in_all_runs": tags_in_all,
37        "tags_in_exactly_one_run": tags_in_exactly_one,
38        "latency_p50_ms": round(p50, 2),
39        "latency_p95_ms": round(p95, 2),
40        "latency_p99_ms": round(p99, 2)
41    }
42
43 results_07 = [r for r in all_results if r["temperature"] == 0.7]
44 results_00 = [r for r in all_results if r["temperature"] == 0.0]
45
46 summary_07 = analyze_temp(results_07, "0.7")
47 summary_00 = analyze_temp(results_00, "0.0")
48
49 with open("../reports/hw01/METRICS.md", "w") as f:
50     f.write("# Non-Deterministic Metrics\n\n")
51     f.write("## Temperature 0.7\n\n")
52     f.write(f"Distinct tag sets: {summary_07['distinct_tag_sets']}\n")
53     f.write(f"Tags in all 20 runs: {summary_07['tags_in_all_runs']}\n")
54     f.write(f"Tags in exactly 1 run: {summary_07['tags_in_exactly_one_run']}\n")
55     f.write(f"Latency p50: {summary_07['latency_p50_ms']} ms\n")
56     f.write(f"Latency p95: {summary_07['latency_p95_ms']} ms\n")
57     f.write(f"Latency p99: {summary_07['latency_p99_ms']} ms\n\n")
58     f.write("## Temperature 0.0\n\n")
59     f.write(f"Distinct tag sets: {summary_00['distinct_tag_sets']}\n")
60     f.write(f"Tags in all 20 runs: {summary_00['tags_in_all_runs']}\n")
61     f.write(f"Tags in exactly 1 run: {summary_00['tags_in_exactly_one_run']}\n")
62     f.write(f"Latency p50: {summary_00['latency_p50_ms']} ms\n")
63     f.write(f"Latency p95: {summary_00['latency_p95_ms']} ms\n")
64     f.write(f"Latency p99: {summary_00['latency_p99_ms']} ms\n")
65
66 print("\nMETRICS.md written to reports/hw01/METRICS.md")

```

```

(venv) C:\Users\vivek\Desktop\Rutuja docs\DATA 260\data260-5825>python run_experiments.py
== Starting 20 runs at temperature 0.7 ==
Running temp=0.7, run 1/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 24714.79ms
Running temp=0.7, run 2/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Phase II Trial'], Latency: 36030.62ms
Running temp=0.7, run 3/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 23105.28ms
Running temp=0.7, run 4/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Phase II Clinical Trial'], Latency: 28072.78ms
Running temp=0.7, run 5/20...
Tags: ['Phase II Trial', 'Metformin', 'Type 2 Diabetes'], Latency: 39078.15ms
Running temp=0.7, run 6/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Phase II Trial'], Latency: 45094.62ms
Running temp=0.7, run 7/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Clinical Trial Phase II'], Latency: 54552.54ms
Running temp=0.7, run 8/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 27791.6ms
Running temp=0.7, run 9/20...
Tags: ['Metformin Efficacy', 'Phase II Clinical Trial', 'Blood Glucose Management'], Latency: 28190.46ms
Running temp=0.7, run 10/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Phase II Trial'], Latency: 48165.82ms
Running temp=0.7, run 11/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Clinical Trial'], Latency: 48429.48ms
Running temp=0.7, run 12/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Clinical Trial Phase II'], Latency: 35527.82ms
Running temp=0.7, run 13/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Clinical Trial Phase II'], Latency: 44432.31ms
Running temp=0.7, run 14/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 25347.13ms
Running temp=0.7, run 15/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Blood Glucose Management'], Latency: 26358.0ms
Running temp=0.7, run 16/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Phase II Trial'], Latency: 45613.92ms
Running temp=0.7, run 17/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 35899.71ms
Running temp=0.7, run 18/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 29978.09ms
Running temp=0.7, run 19/20...
Tags: ['Phase II Trial', 'Metformin Efficacy', 'Type 2 Diabetes'], Latency: 38643.75ms
Running temp=0.7, run 20/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes', 'Phase II Trial'], Latency: 42261.76ms

```

```

=== Starting 20 runs at temperature 0.0 ===
Running temp=0.0, run 1/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Metformin Safety'], Latency: 58469.29ms
Running temp=0.0, run 2/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 32533.61ms
Running temp=0.0, run 3/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33518.69ms
Running temp=0.0, run 4/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30482.58ms
Running temp=0.0, run 5/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33516.66ms
Running temp=0.0, run 6/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30744.5ms
Running temp=0.0, run 7/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33531.37ms
Running temp=0.0, run 8/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30452.11ms
Running temp=0.0, run 9/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33493.14ms
Running temp=0.0, run 10/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30390.39ms
Running temp=0.0, run 11/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33478.53ms
Running temp=0.0, run 12/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30410.33ms
Running temp=0.0, run 13/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33543.15ms
Running temp=0.0, run 14/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30411.64ms
Running temp=0.0, run 15/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33511.32ms
Running temp=0.0, run 16/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30398.42ms
Running temp=0.0, run 17/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33525.61ms
Running temp=0.0, run 18/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30374.8ms
Running temp=0.0, run 19/20...
Tags: ['Phase II Clinical Trial', 'Metformin Efficacy', 'Type 2 Diabetes Management'], Latency: 33481.41ms
Running temp=0.0, run 20/20...
Tags: ['Metformin Efficacy', 'Type 2 Diabetes Management', 'Clinical Trial Safety'], Latency: 30393.76ms
All 40 runs complete. Results saved to reports/hw01/raw/

```

Save the raw per-run results (tags + latency, all 40 runs) as JSON/CSV

↑ ↻ Start backup > Desktop > Rutuja docs > DATA 260 > data260-5825 > reports > hw01 > raw

	Name	Date modified	Type	Size
ds	all_runs	8/30/2026 3:08 PM	Microsoft Excel Co...	10 KB
ts	all_runs	8/30/2026 3:08 PM	JSON Source File	15 KB

Results Summary: Distinct Tags and Latency (Temp 0.7 vs Temp 0.0):

```

(venv) C:\Users\vivek\Desktop\Rutuja docs\DATA 260\data260-5825>python analyze_results.py

=== Temperature 0.7 ===
Distinct tag sets: 6
Tags in all 20 runs: []
Tags in exactly 1 run: ['Metformin', 'Clinical Trial']
Latency p50: 35965.17 ms
Latency p95: 45613.92 ms
Latency p99: 45613.92 ms

=== Temperature 0.0 ===
Distinct tag sets: 3
Tags in all 20 runs: ['Metformin Efficacy']
Tags in exactly 1 run: ['Metformin Safety']
Latency p50: 33006.07 ms
Latency p95: 33543.15 ms
Latency p99: 33543.15 ms

METRICS.md written to reports/hw01/METRICS.md

```

Q. Using your own numbers, explain what two users sending identical input might see,

If two different people typed in the exact same clinical trial info, they might not get the exact same tags back. At the 0.7 temperature, we got 6 different combinations of tags across 20 tries, so two people would likely see different results. At the 0.0 temperature, one tag ("Metformin Efficacy") showed up almost every time, so two people might see the same thing sometimes but not always.

Q. and give one example where run-to-run variation is acceptable and one where it is not.

The variation is acceptable if this feature was just helping people browse and discover similar trials, it wouldn't matter if the tags change a little each time as people would still discover trials that are genuinely relevant to what they're looking for. And when the variation is random it is not acceptable if the same system was used for something official where exact same answer is expected every time, if it keeps changing it can lead to mistakes in official records and hence here more consistent setting of temperature 0.0 should be used instead of the random one.

Model Client and Token Accounting (Part 4):

5-Turn Conversation with Token Tracking and /stats Checkpoints:

```
src > model_client.py > ModelClient > complete
1 from langchain_ollama import ChatOllama
2
3 class ModelClient:
4     def __init__(self, model="qwen3:8b", temperature=0.7):
5         self.llm = ChatOllama(model=model, temperature=temperature)
6
7     def complete(self, messages, tools=None):
8         """
9         messages: list of dicts like [{"role": "system"/"user"/"assistant", "content": "..."}]
10        Returns the model's response text.
11        """
12        response = self.llm.invoke(messages)
13        return response.content
```

```
code > hwl_client.py > main
1 import sys
2 sys.path.append("../src")
3 from model_client import ModelClient
4
5 def count_tokens(text):
6     # Simple approximation: 4 characters per token (common rule of thumb)
7     return max(1, len(text) // 4)
8
9 def main():
10    client = ModelClient()
11    conversation_history = []
12    system_prompt = [{"role": "system", "content": "You are a helpful assistant."}]
13    conversation_history.append(system_prompt)
14
15    cumulative_input_tokens = 0
16    cumulative_output_tokens = 0
17    turn_count = 0
18
19    print("HLL Model Client - type your message, or '/stats' for stats, or 'exit' to quit.\n")
20
21    while True:
22        user_input = input("You: ")
23
24        if user_input.strip().lower() == "exit":
25            break
26
27        if user_input.strip().lower() == "/stats":
28            history_length = sum(len(m["content"]) for m in conversation_history)
29            print(f"--- Stats ---")
30            print(f"turn count: {turn_count}")
31            print(f"Cumulative input tokens: {cumulative_input_tokens}")
32            print(f"Cumulative output tokens: {cumulative_output_tokens}")
33            print(f"Cumulative total tokens: {cumulative_input_tokens + cumulative_output_tokens}")
34            print(f"Serialized conversation history length: {history_length} characters")
35            print(f"-----\n")
36            continue
37
38        conversation_history.append({"role": "user", "content": user_input})
39
40        input_text_this_turn = " ".join(m["content"] for m in conversation_history)
41        input_tokens = count_tokens(input_text_this_turn)
42
43        response_text = client.complete(conversation_history)
44
45        output_tokens = count_tokens(response_text)
46
47        conversation_history.append({"role": "assistant", "content": response_text})
48
49        turn_count += 1
50        cumulative_input_tokens += input_tokens
51        cumulative_output_tokens += output_tokens
52
53        print(f"Assistant: {response_text}")
54        print(f"[turn {turn_count}] - Input tokens: {input_tokens}, Output tokens: {output_tokens}, Total: {input_tokens + output_tokens}\n")
55
56    print(f"==== Session ended ===")
57    print(f"Total turns: {turn_count}")
58    print(f"Cumulative input tokens: {cumulative_input_tokens}")
59    print(f"Cumulative output tokens: {cumulative_output_tokens}")
60    print(f"Cumulative total tokens: {cumulative_input_tokens + cumulative_output_tokens}")
61
62 if __name__ == "__main__":
63     main()
```

```
(cmd) C:\Users\vivah\Desktop\stats\data\DATA 260\data260-3825python hel_client.py
D:\Python Client - Type your message, or '/stats' for stats, or 'exit' to quit.

You: What is a clinical trial?

Assistant: A clinical trial is a research study designed to evaluate the safety, effectiveness, and potential benefits of a new medical intervention, such as a drug, device, therapy, or treatment. These trials are essential for advancing medical knowledge and improving healthcare by testing whether new interventions work better than existing ones or are safer for patients.

### Key Aspects of Clinical Trials:

1. Objectives:
   - Primary: Assess whether a treatment is safe for human use.
   - Secondary: Determine if a treatment works as intended.
   - Optimal Use: Identify the best dosage, administration method, or combination with other treatments.

2. Phases of Trials:
   - Clinical trials are divided into "phases" to systematically evaluate interventions:
     - Phase I: Small groups of healthy volunteers or patients to test safety, side effects, and dosage.
     - Phase II: Larger groups to assess effectiveness and monitor side effects.
     - Phase III: Large-scale trials to confirm effectiveness, monitor long-term side effects, and compare the intervention to standard treatments.
     - Phase IV: Post-approval studies to track long-term outcomes and rare side effects after the treatment is available to the public.

3. Types of Trials:
   - Randomized Controlled Trials (RCTs): Participants are randomly assigned to treatment or control groups to ensure unbiased results.
   - Blind/Double-Blind Trials: Participants (and sometimes researchers) don't know who received the treatment to avoid bias.
   - Observational Studies: Researchers observe and record data without intervening (e.g., tracking outcomes in real-world settings).

4. Ethical and Regulatory Oversight:
   - Trials require "informed consent", ensuring participants understand risks, benefits, and their right to withdraw.
   - They are reviewed by ethics committees (e.g., Institutional Review Boards or IRBs) to protect participant safety and rights.

5. Benefits and Risks:
   - Benefits: Access to cutting-edge treatments, potential health improvements, and contribution to medical research.
   - Risks: Possible side effects, time commitment, or discomfort.

6. Examples:
   - Testing a new cancer drug.
   - Evaluating a vaccine for a public health crisis (e.g., COVID-19).
   - Studying a novel surgical technique or device.

### Why Clinical Trials Matter:
They are the foundation of evidence-based medicine, enabling the development of life-saving treatments and ensuring that new therapies are both safe and effective before they are widely used. Participation in trials also allows researchers to address unmet medical needs and improve patient care.

If you're considering participating in a clinical trial, it's important to discuss the details with your healthcare provider and the research team to understand the risks and benefits.

[Turn 1 - Input tokens: 13, Output tokens: 735, Total: 748]

You: What does Phase II mean in that context?

Assistant: Phase II of a clinical trial is a critical stage in the development of a new medical intervention (e.g., drug, device, or therapy). It follows Phase I and precedes Phase III, serving as a bridge between initial safety testing and large-scale efficacy evaluation. Here's a breakdown of what Phase II entails:

---

### Purpose of Phase II:

1. Assess Efficacy:
   - Evaluate whether the intervention shows promising efficacy and acceptable safety.
   - Statistical significance is evaluated to assess whether results are likely due to the treatment rather than chance.

---

### Key Features of Phase II:

- Participants:
  - Focuses on patients with the target condition (not healthy volunteers).
  - Typically involves 100-300 participants.

- Durations:
  - Usually lasts 4-12 months, depending on the intervention and study design.

- Study Designs:
  - May include randomized controlled trials (RCTs) or observational studies, depending on the research goals.
  - Often uses "double-blind" or standard treatments as controls to compare outcomes.

- Outcomes:
  - Researchers analyze data to determine if the intervention shows promising efficacy and acceptable safety.
  - Statistical significance is evaluated to assess whether results are likely due to the treatment rather than chance.

---

### How Phase II Differs from Phase I and III:

- Phase I: Focuses on safety and dosage in a small group of healthy volunteers or patients.
- Phase II: Balances efficacy and safety in a larger group of patients.
- Phase III: Confirms efficacy and safety in a "much larger population" (often thousands of participants) to gather robust data for regulatory approval.

---

### Examples:

If a new diabetes drug is tested in Phase II:
- Researchers might evaluate whether it effectively lowers blood sugar levels in patients with type 2 diabetes.
- They would also track side effects like hypoglycemia or gastrointestinal issues.
- If the drug shows promise, it would advance to Phase III for broader testing.

---

### Why Phase II Matters:
Phase II is where researchers "validate the potential of an intervention" and decide whether it's worth investing further resources (e.g., for Phase III trials). It helps identify treatments that are both safe and effective before they are widely tested or approved for public use.

Let me know if you'd like examples of specific drugs or conditions tested in Phase II!

[Turn 2 - Input tokens: 759, Output tokens: 783, Total: 1522]

You: Summarize that in one sentence.

Assistant: Phase II clinical trials evaluate the efficacy and safety of a new treatment in a larger group of patients (typically 100-300 participants) to determine if it shows promise and should advance to large-scale testing in Phase III.

[Turn 3 - Input tokens: 133, Output tokens: 57, Total: 188]

You: /stats

--- STATS ---
Turn count: 3
Cumulative input tokens: 2385
Cumulative output tokens: 1355
Cumulative total tokens: 3740
Serialized conversation history length: 6368 characters
-----
```

/stats output after turn 3

```
### Key Features of Phase II:

- Participants:
  - Focuses on patients with the target condition (not healthy volunteers).
  - Typically involves 100-300 participants.

- Durations:
  - Usually lasts 4-12 months, depending on the intervention and study design.

- Study Designs:
  - May include randomized controlled trials (RCTs) or observational studies, depending on the research goals.
  - Often uses "double-blind" or standard treatments as controls to compare outcomes.

- Outcomes:
  - Researchers analyze data to determine if the intervention shows promising efficacy and acceptable safety.
  - Statistical significance is evaluated to assess whether results are likely due to the treatment rather than chance.

---

### How Phase II Differs from Phase I and III:

- Phase I: Focuses on safety and dosage in a small group of healthy volunteers or patients.
- Phase II: Balances efficacy and safety in a larger group of patients.
- Phase III: Confirms efficacy and safety in a "much larger population" (often thousands of participants) to gather robust data for regulatory approval.

---

### Examples:

If a new diabetes drug is tested in Phase II:
- Researchers might evaluate whether it effectively lowers blood sugar levels in patients with type 2 diabetes.
- They would also track side effects like hypoglycemia or gastrointestinal issues.
- If the drug shows promise, it would advance to Phase III for broader testing.

---

### Why Phase II Matters:
Phase II is where researchers "validate the potential of an intervention" and decide whether it's worth investing further resources (e.g., for Phase III trials). It helps identify treatments that are both safe and effective before they are widely tested or approved for public use.

Let me know if you'd like examples of specific drugs or conditions tested in Phase II!

[Turn 4 - Input tokens: 759, Output tokens: 783, Total: 1522]

You: Summarize that in one sentence.

Assistant: Phase II clinical trials evaluate the efficacy and safety of a new treatment in a larger group of patients (typically 100-300 participants) to determine if it shows promise and should advance to large-scale testing in Phase III.

[Turn 5 - Input tokens: 133, Output tokens: 57, Total: 188]

You: /stats

--- STATS ---
Turn count: 5
Cumulative input tokens: 5630
Cumulative output tokens: 1710
Cumulative total tokens: 7340
Serialized conversation history length: 7820 characters
-----
```

/stats output after turn 5

```
You: What is Metformin used for?

Assistant: Metformin is primarily used to treat type 2 diabetes by improving insulin sensitivity, reducing glucose production in the liver, and increasing glucose uptake in muscle cells. It is also sometimes prescribed "off-label" for conditions like polycystic ovary syndrome (PCOS) to manage insulin resistance and obesity (though its role in weight loss is less well-established). It is generally considered a first-line therapy for type 2 diabetes due to its efficacy, affordability, and low risk of hypoglycemia.

[Turn 4 - Input tokens: 155, Output tokens: 131, Total: 1726]

You: Thank you, that's all.

Assistant: You're welcome! If you have any more questions in the future, feel free to ask. Have a great day! 🌟

[Turn 5 - Input tokens: 1732, Output tokens: 24, Total: 1756]

You: /stats

--- STATS ---
Turn count: 5
Cumulative input tokens: 5630
Cumulative output tokens: 1710
Cumulative total tokens: 7340
Serialized conversation history length: 7820 characters
-----
```

You: exit

=== Session ended ===

Total turns: 5

Cumulative input tokens: 5630

Cumulative output tokens: 1710

Cumulative total tokens: 7340

AGENT.md verification test output

```
1_client.py code 1 | verify.py code | test_agent_review.py code 1 | run_experiments.py code
AGENT.md > abc # Agent Instructions
1 # Agent Instructions
2
3 When asked to review code, you MUST respond using ONLY bullet points.
4 Do not include any introductory sentences, paragraphs, explanations outside
5 of bullets, or concluding remarks. Every piece of feedback must be a
6 single bullet point starting with "- ".
7
8 Do not use headings, code blocks, or numbered lists – bullet points only.
```

```
(venv) C:\Users\vivek\Desktop\Rutuja docs\DATA 260\data260-5825>python test_agent_review.py
=== Model Response ===
- The function is functionally correct for basic addition but lacks type hints for parameters and return value.
- Parameter names 'a' and 'b' are generic; more descriptive names would improve readability.
- The intermediate variable 'result' is unnecessary; the function could directly return 'a + b'.
- No error handling is included for non-numeric inputs, which could cause runtime errors.
- The function could benefit from a docstring to clarify its purpose and usage.
```

Questions:

1. why is prior conversation context resent with every turn?
The AI has no memory. Everytime a message is sent the AI forgets everything the second the conversation ends. Hence to remember the conversation, the app must resend the full conversation every time just to remind it what's already been said.
2. How is a system prompt different from a user message?
The system prompt is the fixed rule that is set at once for how the AI should behave whereas the user message is the actual question or task you ask it to do and it will change every time.
3. Why do input tokens grow over a conversation?
To remind AI the full conversation we have to resend it every time and hence it keeps getting bigger and bigger the more we talk. This is what my numbers showed, going from 13 tokens on turn 1 to over 1700 by turn 5.
4. What eventually limits that growth?
Every AI model has a maximum size limit for how much text it can read at once, so eventually a long conversation would hit that limit and either break or need older messages trimmed out.

Self-Check Verification Output: Output of verify.py, confirming all required files are present, Python version is compatible, Ollama with the local model is available, and all 40 non-determinism runs completed successfully.

```

1 client.py code 1 verify.py code X test_agent_review.py code 1 run_experiments.py code analyze_results.py code
code > verify.py > ...
1 import json
2 import os
3 import subprocess
4 import sys
5
6 checks = {}
7
8 # Check 1: Required files exist (paths relative to code/ folder)
9 required_files = [
10     "web_application/index.html", "web_application/script.js", "web_application/Dockerfile",
11     "agents_demo.py", "../src/model_client.py", "hwl_client.py", "../AGENT.md",
12     "../ODRAIN_SCHEMA.md", "../reports/hwll/cases/non_determination_input.json",
13     "../reports/hwll/rau/all_runs.json", "../reports/hwll/WEIRIC.md",
14     "../reports/hwll/AI_USE.md", "../reports/hwll/RLR_100.txt"
15 ]
16 missing_files = [f for f in required_files if not os.path.exists(f)]
17 checks["required_files_present"] = len(missing_files) == 0
18 checks["missing_files"] = missing_files
19
20 # Check 2: Python version
21 python_version = sys.version_info
22 checks["python_version"] = f'{python_version.major}.{python_version.minor}.{python_version.micro}'
23 checks["python_version_ok"] = python_version.major == 3 and python_version.minor in [11, 12]
24
25 # Check 3: Ollama model available
26 try:
27     result = subprocess.run(["ollama", "list"], capture_output=True, text=True, timeout=10)
28     checks["ollama_available"] = "qwen3:8b" in result.stdout
29 except Exception:
30     checks["ollama_available"] = False
31
32 # Check 4: 40 runs completed
33 if os.path.exists("../reports/hwll/rau/all_runs.json"):
34     with open("../reports/hwll/rau/all_runs.json") as f:
35         runs = json.load(f)
36     checks["total_runs_count"] = len(runs)
37     checks["forty_runs_complete"] = len(runs) == 40
38 else:
39     checks["total_runs_count"] = 0
40     checks["forty_runs_complete"] = False
41
42 checks["overall_status"] = "PASS" if (
43     checks["required_files_present"] and
44     checks["python_version_ok"] and
45     checks["ollama_available"] and
46     checks["forty_runs_complete"]
47 ) else "FAIL"
48
49 with open("../reports/hwll/verification.json", "w") as f:
50     json.dump(checks, f, indent=2)
51
52 print(json.dumps(checks, indent=2))

```

```

(venv) C:\Users\vivek\Desktop\Rutuja docs\DATA 260\data260-5825>python verify.py
{
  "required_files_present": true,
  "missing_files": [],
  "python_version": "3.11.9",
  "python_version_ok": true,
  "ollama_available": true,
  "total_runs_count": 40,
  "forty_runs_complete": true,
  "overall_status": "PASS"
}

```